

Software Requirements Specification

for

Bank Management System

Version 1.0

Prepared by :

Abhishek Hanamant Kinagi (PES1UG24CS018),
Adithya Ajeeth (PES1UG24CS022),
Aditya M (PES1UG24CS027) and
Aditya Vats (PES1UG24CS032)

PES UNIVERSITY : SOFTWARE ENGINEERING (UE24CS341A)

Created : 02/09/2026

Table of Contents

Revision History

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 Product Scope
- 1.5 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. External Interface Requirements

- 3.1 User Interfaces
- 3.2 Hardware Interfaces
- 3.3 Software Interfaces
- 3.4 Communications Interfaces

4. System Features

- 4.1 User Authentication and Session Management
- 4.2 Customer Record Management
- 4.3 Account Management
- 4.4 Deposit and Withdrawal Transactions
- 4.5 Fund Transfer
- 4.6 Balance Inquiry and Transaction History
- 4.7 Interest Computation and Account Statements
- 4.8 Staff Management and Role Dashboards

5. Other Nonfunctional Requirements

- 5.1 Performance Requirements
- 5.2 Safety Requirements
- 5.3 Security Requirements
- 5.4 Software Quality Attributes
- 5.5 Business Rules

6. Other Requirements

Appendix A: Glossary

Appendix B: Analysis Models

Appendix C: To Be Determined List

Appendix D: Future Enhancements

Revision History

Date	Reason For Changes	Version
02/09/2026	Initial SRS release	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements for Version 1.0 of the Bank Management System, an application for managing customer accounts and day-to-day banking operations including deposits, withdrawals, fund transfers, balance inquiry, and transaction history. The system consists of a web-based front end and a back-end core, implemented in C/C++, that holds all account and transaction processing logic. This document covers the complete scope of the system as developed for this Software Engineering mini-project. There are no separate subsystems or partial-system SRS documents associated with this project.

1.2 Document Conventions

Throughout this document, the keyword “shall” denotes a mandatory requirement (e.g., “The system shall ...”). Functional requirements are uniquely identified using the prefix FR- (e.g., FR-1, FR-2); non-functional requirements are identified using the prefix NFR- (e.g., NFR-1, NFR-2). Each System Feature in Section 4 is assigned a priority of High, Medium, or Low to indicate its relative importance; unless stated otherwise, the functional requirements within a feature share that feature’s priority. All monetary amounts are expressed in Indian Rupees (INR).

1.3 Intended Audience and Reading Suggestions

This document is intended for the members of the project team, the course instructor/evaluator, and any other reviewer of the project. Readers seeking a general understanding of the system should read Sections 1 and 2. Developers and testers should focus on Sections 3, 4, and 5, which contain the external interfaces, the feature-level functional requirements, and the non-functional requirements respectively. The appendices are provided for reference and may be consulted as needed.

1.4 Product Scope

The Bank Management System is a simplified, single-branch banking application that maintains customer records, bank accounts, and the transactions performed against those accounts.

The system will allow:

- Administrators to manage bank staff accounts, define account types and their parameters (minimum balance and interest rate), and view system-wide summary information.
- Bank Officers to register customers, open and close accounts, and process deposits, withdrawals, and fund transfers on behalf of customers.
- Customers to view their own profile, accounts, current balances, and transaction history, and to obtain an account statement for a selected period.

The system will provide role-specific dashboards summarizing relevant information for each user type, and will maintain an auditable record of every balance-changing operation.

The following are explicitly out of scope for Version 1.0 and are listed only under Appendix D (Future Enhancements) where applicable: loan and EMI management, fixed and recurring deposits, cheque processing and clearing, ATM or debit card operations, internet and mobile banking channels, interbank transfers (NEFT/RTGS/UPI), multi-branch and multi-currency operation, and joint accounts.

1.5 References

- IEEE Std. 830-1998, IEEE Recommended Practice for Software Requirements Specifications, Institute of Electrical and Electronics Engineers, 1998.
- Software Engineering course material and SRS template (UE23CS341A), Department of CSE — slide deck provided by the course instructor, current academic term.
- Wiegers, Karl E., Software Requirements Specification template, 1999 (used as the structural template for this document; distributed for use and modification by the author).

2. Overall Description

2.1 Product Perspective

The Bank Management System is a self-contained, standalone product. It is not a replacement for, or an extension of, any existing banking software, and it does not interface with any real financial network. The system follows a client-server architecture with three logical layers: a browser-based presentation layer, a back-end core implemented in C/C++ that owns all account and transaction processing logic, and a relational database used for persistent storage. All balance-changing operations are performed by the C/C++ core; the front end is a presentation and data-entry layer only and contains no banking logic of its own. The system is developed as an academic project to demonstrate the application of Software Engineering principles including requirements engineering, design, implementation, testing, and version-controlled collaborative development.

2.2 Product Functions

At a high level, the system provides the following functions:

- Secure login and role-based access for Administrators, Bank Officers, and Customers.
- Registration and maintenance of customer records by the Administrator or a Bank Officer.
- Opening, freezing, and closing of bank accounts, each linked to a registered customer and a defined account type.
- Processing of deposit and withdrawal transactions against active accounts, subject to balance and limit validation.
- Transfer of funds between two accounts held within the bank, applied as a single atomic operation.
- Balance inquiry and viewing of session-wise transaction history by Customers and Bank Officers.
- Computation and crediting of interest for eligible account types, and generation of account statements for a selected period.
- Management of bank staff accounts by the Administrator.
- Role-specific dashboards providing a summary view relevant to each user type.
- Maintenance of an audit trail recording every balance-changing operation together with the operator responsible for it.

2.3 User Classes and Characteristics

User Class	Characteristics
Administrator	Assumed to be a semi-technical staff member responsible for system-wide configuration and oversight. Manages bank staff accounts and account-type parameters, and views summary reports. Does not routinely perform customer transactions. Has the highest privilege level in the system.
Bank Officer	Assumed to have basic computer literacy and to be the primary day-to-day user of the system. Requires fast, form-based screens to register customers, open accounts, and process deposits, withdrawals, and transfers. Access is scoped to customer-facing operations; cannot manage staff accounts or system configuration.
Customer	Assumed to have basic computer literacy. Primarily requires read-only, easy-to-navigate views of their own accounts, balances, and transaction history.

User Class	Characteristics
	Access is scoped to their own records only and does not include any operation that modifies another party's balance.

2.4 Operating Environment

The system is deployed in a client-server configuration. The client side requires a modern web browser (e.g., Chrome, Firefox) running on a standard desktop or laptop computer; no native mobile application is required. The server side requires an environment capable of compiling and running the C/C++ back-end core — a standards-compliant C/C++ toolchain (e.g., GCC/G++ or MSVC) on Windows 10/11 or a mainstream Linux distribution — together with a relational database management system. The exact hosting environment (local development machine, cloud, or institutional server) is left to the discretion of the development team for this academic project.

2.5 Design and Implementation Constraints

- The system is developed as an academic mini-project by a team of four students within a limited timeframe and is scoped accordingly.
- The core back-end functionality — account management, transaction processing, validation, and interest computation — shall be implemented in C/C++, as required by the assigned project topic. The front end may be implemented using standard web technologies at the team's discretion.
- The front end shall contain no banking business logic; all balance-changing decisions shall be taken by the C/C++ core, so that the same rules apply regardless of the client used.
- The system supports only three user roles — Administrator, Bank Officer, and Customer — in Version 1.0; no additional roles (e.g., branch manager, auditor) are supported.
- The system is a single-branch, single-currency (INR) system and does not support multi-tenancy or multi-branch operation.
- The exact technology stack (front-end framework, C/C++ web/API layer, and database product) is not finalized at the time of writing this document and is left to the development team's discretion, provided it supports the stated requirements.
- The system should follow a layered architecture (separating presentation, business logic, and data storage concerns) to the extent practical, to support maintainability and ease of testing; the specific architectural pattern and its degree of separation are left to the development team's judgment.
- The system shall use a relational database to preserve referential integrity between related entities (e.g., customers, accounts, transactions) and to support transactional consistency for balance updates.
- The system is accessed through standard modern web browsers; no native mobile application is provided.
- The project shall be developed using Git for version control, with collaborative development managed through a shared GitHub repository, as required by the course.

2.6 User Documentation

No formal end-user manual is planned for Version 1.0, given the scope of an academic mini-project. Basic in-application guidance (field labels, tooltips, and confirmation/error messages) shall be provided in place of a separate manual. Project documentation — including this SRS, design

documents, build instructions for the C/C++ back end, and a README — shall be maintained in the project’s GitHub repository for reference by the team and evaluators.

2.7 Assumptions and Dependencies

- It is assumed that all user accounts (Administrator, Bank Officer, Customer) are created and provisioned from within the system by an authorised user; there is no public self-registration.
- It is assumed that a customer may hold more than one account, and that each account belongs to exactly one customer; joint accounts are not supported in Version 1.0.
- It is assumed that each account is associated with exactly one account type (e.g., Savings, Current), and that the minimum balance and interest rate applicable to that type are defined when the type is set up.
- It is assumed that the physical handling of cash occurs outside the system, and that the system is the authoritative record of the resulting balances rather than a control mechanism for any cash-handling device.
- It is assumed that the server’s system clock is correct, since transaction timestamps and statement periods depend on it.
- It is assumed that a working internet or local network connection and a modern web browser are available to all users.
- The system depends on the availability of a relational database for persistent storage of customer, account, and transaction records.
- The system depends on the availability of a standards-compliant C/C++ toolchain on the server for building and running the back-end core.

3. External Interface Requirements

3.1 User Interfaces

The system shall provide a web-based graphical user interface accessible through standard web browsers. It shall provide distinct, role-specific views for Administrator, Bank Officer, and Customer users, including a login screen, dashboards, and forms for data entry and viewing, following consistent navigation and layout conventions across all screens. Password and PIN fields shall be masked on entry. Every operation that changes an account balance shall present a confirmation step showing the account, the amount, and the resulting balance before the operation is committed, and shall display a clear success or failure message afterwards.

3.2 Hardware Interfaces

The system does not require any specialized hardware. It shall run on standard computing devices (desktop/laptop) capable of running a modern web browser on the client side and the C/C++ back-end core on the server side. No integration with cash dispensers, card readers, passbook printers, or other banking peripherals is required in Version 1.0.

3.3 Software Interfaces

The C/C++ back-end core shall interface with a relational database management system for data storage and retrieval, using a database driver or connector appropriate to the chosen database product. The back-end shall expose a well-defined API layer over standard web protocols (e.g., HTTP/HTTPS), which the front end shall consume; data exchanged across this interface shall use a standard structured format (e.g., JSON). No interface to any external financial system, payment network, or third-party banking service is required in Version 1.0.

3.4 Communications Interfaces

Where the system is deployed to a network-accessible environment, all client-server communication shall use HTTPS to protect credentials, account numbers, and balance information in transit. In local development and testing environments, plain HTTP may be used for convenience, provided no real customer or financial data is transmitted. The system requires no email, messaging, or interbank communication interfaces in Version 1.0.

4. System Features

4.1 User Authentication and Session Management (Priority: High)

4.1.1 Description and Priority

Provides secure login for all user roles, enforces role-based access to system functionality, and controls session termination. Priority: High.

4.1.2 Stimulus/Response Sequences

- User enters valid credentials and submits the login form → System authenticates the user and redirects to the role-appropriate dashboard.
- User enters invalid credentials → System displays a generic error message, does not grant access, and increments the failed-attempt count for that account.
- User enters invalid credentials three consecutive times → System locks the account and directs the user to contact an Administrator.
- User selects “Logout” → System terminates the active session and returns the user to the login screen.

4.1.3 Functional Requirements

ID	Requirement
FR-1	The system shall allow registered users (Administrator, Bank Officer, Customer) to log in using a user identifier and a password/PIN, and shall redirect each user to a dashboard appropriate to their role. The system shall restrict access to features and data based on the logged-in user’s role (RBAC), and shall enforce this restriction in the back-end core rather than relying on the front end to hide unavailable options.
FR-2	The system shall allow a logged-in user to log out, terminating their active session. The system shall lock a user account after three consecutive failed login attempts; a locked account shall be re-enabled only by an Administrator.

4.2 Customer Record Management (Priority: High)

4.2.1 Description and Priority

Allows the Administrator or a Bank Officer to maintain the master list of customer records held by the bank. Priority: High.

4.2.2 Stimulus/Response Sequences

- Bank Officer submits a new customer’s details → System validates the details, creates a new customer record, and assigns a unique customer identifier.
- Bank Officer selects an existing customer and edits their details → System updates the corresponding record.
- Administrator deactivates a customer → System marks the record inactive and prevents any new account from being opened for that customer.
- User submits a record with a missing or invalid mandatory field → System rejects the submission and indicates the specific field in error.

4.2.3 Functional Requirements

ID	Requirement
FR-3	The system shall allow the Administrator or a Bank Officer to create, view, update, and deactivate customer records, including basic personal and contact details (e.g., name, date of birth, address, phone number, and identification reference). Each customer shall be assigned a unique customer identifier on creation. Deactivated customers shall be retained in the system but shall be excluded from the opening of new accounts.

4.3 Account Management (Priority: High)

4.3.1 Description and Priority

Allows a Bank Officer to open accounts for registered customers, and to freeze or close existing accounts. Priority: High.

4.3.2 Stimulus/Response Sequences

- Bank Officer selects a registered customer, chooses an account type, and enters an initial deposit → System creates the account, assigns a unique account number, and records the initial deposit as the account's first transaction.
- Bank Officer attempts to open an account with an initial deposit below the minimum balance for the chosen type → System rejects the request and displays the required minimum.
- Administrator freezes an account → System marks the account frozen and rejects all subsequent deposits, withdrawals, and transfers against it while permitting balance and history viewing.
- Bank Officer requests closure of an account with a zero balance → System closes the account and retains its transaction history.
- Bank Officer requests closure of an account with a non-zero balance → System rejects the closure and prompts for the balance to be withdrawn or transferred first.

4.3.3 Functional Requirements

ID	Requirement
FR-4	The system shall allow a Bank Officer to open a new account for an active registered customer, capturing the account type and an initial deposit. The system shall assign a unique account number to each account and shall reject an initial deposit lower than the minimum balance defined for the selected account type.
FR-5	The system shall allow an authorised user to freeze, unfreeze, and close an account. A frozen account shall accept no deposit, withdrawal, or transfer while remaining viewable. An account may be closed only when its balance is zero; closed and frozen accounts and their transaction history shall be retained and shall never be permanently deleted.

4.4 Deposit and Withdrawal Transactions (Priority: High)

4.4.1 Description and Priority

Allows a Bank Officer to credit and debit an active account, and records each such operation in the transaction history. Priority: High.

4.4.2 Stimulus/Response Sequences

- Bank Officer selects an active account, enters a deposit amount, and confirms → System credits the balance and records a deposit transaction with a timestamp and the operator's identifier.
- Bank Officer selects an active account, enters a withdrawal amount within the available balance, and confirms → System debits the balance and records a withdrawal transaction.
- Bank Officer enters a withdrawal that would take the balance below the minimum balance for the account type → System rejects the transaction, leaves the balance unchanged, and states the reason.
- Bank Officer enters a withdrawal that would exceed the configured daily withdrawal limit for the account → System rejects the transaction and states the remaining limit.
- Bank Officer enters a zero, negative, or non-numeric amount → System rejects the input before any balance is modified.

4.4.3 Functional Requirements

ID	Requirement
FR-6	The system shall allow a Bank Officer to deposit a positive amount into an active account, updating the account balance and recording a corresponding transaction entry containing the amount, type, timestamp, resulting balance, and the identifier of the operator who performed it.
FR-7	The system shall allow a Bank Officer to withdraw a positive amount from an active account, provided the resulting balance does not fall below the minimum balance defined for the account type and the amount does not exceed the configured daily withdrawal limit. Where a withdrawal is rejected, the system shall leave the account balance unchanged and shall display the specific reason for rejection.

4.5 Fund Transfer (Priority: High)

4.5.1 Description and Priority

Allows funds to be moved between two accounts held within the bank as a single, indivisible operation. Priority: High.

4.5.2 Stimulus/Response Sequences

- Bank Officer selects a source account and a destination account, enters an amount, and confirms → System debits the source, credits the destination, and records two linked transaction entries sharing a common transfer reference.
- Source and destination account are the same → System rejects the transfer.
- Source account has insufficient available balance, or either account is frozen or closed → System rejects the transfer and neither balance is modified.
- A failure occurs after the source has been debited but before the destination is credited → System reverses the debit so that neither account is left in a modified state.

4.5.3 Functional Requirements

ID	Requirement
FR-8	The system shall allow a transfer of a positive amount from one active account to a different active account within the bank. The debit and the credit shall be applied as

ID	Requirement
	a single atomic operation: either both take effect or neither does. The system shall record the transfer as two linked transaction entries, one against each account, sharing a common transfer reference.

4.6 Balance Inquiry and Transaction History (Priority: High)

4.6.1 Description and Priority

Allows Customers to view their own accounts, balances, and transaction history, and Bank Officers to perform the same lookup on behalf of a customer. Priority: High.

4.6.2 Stimulus/Response Sequences

- Customer opens the accounts view → System displays each of the customer's accounts with its account number, type, status, and current balance.
- Customer opens the history view for one of their accounts → System displays the transaction entries for that account in reverse chronological order.
- Customer applies a date range filter → System displays only the transactions falling within that range.
- Customer attempts to view an account they do not own → System denies the request and records the attempt.

4.6.3 Functional Requirements

ID	Requirement
FR-9	The system shall allow a Customer to view the current balance of each account they hold, and shall allow a Bank Officer to view the current balance of any account in the bank.
FR-10	The system shall allow a Customer to view the transaction history of each account they hold, displayed in reverse chronological order and filterable by date range, with each entry showing the date and time, transaction type, amount, and resulting balance. The system shall prevent any Customer from viewing an account they do not hold.

4.7 Interest Computation and Account Statements (Priority: Medium)

4.7.1 Description and Priority

Computes and credits interest for eligible account types and produces account statements for a selected period. Priority: Medium.

4.7.2 Stimulus/Response Sequences

- Administrator triggers the interest run for a period → System computes interest for every eligible active account using the rate defined for its account type, credits it, and records an interest transaction against each account.
- Administrator re-triggers the interest run for a period already processed → System rejects the request to prevent duplicate crediting.
- Customer requests a statement for a selected period → System generates a statement listing the opening balance, all transactions in the period, and the closing balance.

- Customer requests a statement for a period with no transactions → System produces a statement showing the unchanged balance and no entries.

4.7.3 Functional Requirements

ID	Requirement
FR-11	The system shall compute and credit interest for accounts of interest-bearing types, using the interest rate defined for the account type, and shall record each interest credit as a transaction entry against the account. The system shall prevent interest from being credited more than once for the same account and period.
FR-12	The system shall generate an account statement for a selected account and date range, showing the opening balance, each transaction in the period, and the closing balance, and shall allow the statement to be exported in a printable or downloadable form.

4.8 Staff Management and Role Dashboards (Priority: Medium)

4.8.1 Description and Priority

Allows the Administrator to manage bank staff accounts, and provides every user with a role-appropriate summary view on login. Priority: Medium.

4.8.2 Stimulus/Response Sequences

- Administrator submits a new Bank Officer's details → System creates the staff account and issues initial credentials.
- Administrator deactivates a Bank Officer → System marks the account inactive, prevents further logins, and retains the officer's identifier on all past transaction records.
- Administrator logs in → System displays a dashboard summarising counts of customers, accounts, and staff, along with the day's transaction totals.
- Bank Officer logs in → System displays a dashboard summarising the transactions they have processed in the current session or day, with quick links to the transaction screens.
- Customer logs in → System displays a dashboard summarising their accounts and balances, with links to detailed history and statement views.

4.8.3 Functional Requirements

ID	Requirement
FR-13	The system shall allow the Administrator to create, view, update, and deactivate Bank Officer accounts, and to unlock accounts that have been locked by failed login attempts. Deactivated staff accounts shall be retained so that historical transaction records remain attributable.
FR-14	The system shall provide a role-specific dashboard for each user upon login, presenting a brief summary relevant to that user's role (e.g., record and transaction counts for the Administrator, recent activity for a Bank Officer, accounts and balances for a Customer) along with quick links to the corresponding detailed views.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

ID	Requirement
NFR-1	The system shall complete any single balance-affecting transaction (deposit, withdrawal, or transfer), including its persistence to the database, within 2 seconds under normal usage conditions with a dataset of up to a few thousand accounts.
NFR-2	The system shall load standard pages (e.g., dashboards, account lists, transaction history) within 3 seconds under the same conditions.

5.2 Safety Requirements

The Bank Management System does not control any physical equipment or safety-critical process. The corresponding concern for this system is financial data integrity: an incorrect or partially applied transaction constitutes a loss to the user, and the following requirement applies.

ID	Requirement
NFR-3	The system shall never leave an account in a partially updated state. Every balance change shall be applied together with its transaction record as a single atomic unit of work, and shall be rolled back in full if any part of it fails. The system shall never permit an account balance to become negative or to fall below the minimum balance defined for its account type through any operation.

5.3 Security Requirements

ID	Requirement
NFR-4	The system shall store user passwords and PINs in a securely hashed form and shall not display, log, or transmit them in plain text. Access to data and functionality shall be restricted according to the authenticated user's role, and this restriction shall be enforced in the back-end core on every request.
NFR-5	The system shall maintain an audit trail in which every balance-changing operation is recorded with its timestamp, the accounts affected, the amount, and the identifier of the user who performed it. Audit records shall be append-only and shall not be editable or deletable through the application by any role.

5.4 Software Quality Attributes

ID	Attribute	Requirement
NFR-6	Usability	A first-time user in each role shall be able to complete their core task (e.g., a Bank Officer processing a deposit, a Customer viewing their transaction history) without external help. This shall be verified through a usability walkthrough in which at least three representative users each complete the relevant core task on their first attempt.
NFR-7	Reliability	The system shall handle invalid user input (e.g., empty required fields, non-numeric or negative amounts, unknown account numbers) gracefully by displaying clear error messages without crashing, and without modifying any stored balance or record.

ID	Attribute	Requirement
NFR-8	Maintainability	The back-end codebase shall be modular and organised by functional area (e.g., authentication, customer management, account management, transactions, reporting) to facilitate independent development, debugging, and future enhancement by different team members.
NFR-9	Portability	The C/C++ back-end core shall build and run on at least two platforms (e.g., Linux and Windows) using standard language features, without platform-specific code paths in the business logic. The web-based front end shall function correctly on current versions of at least two major web browsers (e.g., Chrome and Firefox) without browser-specific modifications.

5.5 Business Rules

- Only the Administrator may create, modify, or deactivate bank staff accounts and account-type parameters.
- Only a Bank Officer or the Administrator may register customers, open or close accounts, or process deposits, withdrawals, and transfers; a Customer may never initiate an operation that changes a balance in Version 1.0.
- A Customer may view only the accounts they hold, and only the transactions belonging to those accounts.
- A withdrawal or outgoing transfer shall never reduce an account balance below the minimum balance defined for that account type.
- An account may be closed only when its balance is zero; otherwise it may only be frozen.
- Customer, account, and staff records shall only be deactivated or closed, never permanently deleted, in order to preserve the integrity of the transaction history and audit trail.
- Every balance change shall generate exactly one transaction record; a fund transfer generates exactly two linked records sharing a common transfer reference.
- Interest shall be credited at most once per account per interest period.

6. Other Requirements

- Database requirement: the system shall use a relational database management system (RDBMS) to store all persistent data, in order to preserve referential integrity across related entities (customers, accounts, account types, transactions, and staff users) and to provide transactional guarantees for balance updates.
- Implementation language requirement: the core back-end functionality of the system shall be implemented in C/C++, as required by the assigned project topic.
- Version control requirement: all source code and project documentation shall be developed under Git version control, with collaborative development managed through a shared GitHub repository, as required by the course.
- No internationalization, localization, or external regulatory compliance requirements apply to this academic project. The system is not intended for use with real customer funds and makes no claim of compliance with any banking regulation.

Appendix A: Glossary

Term	Description
SRS	Software Requirements Specification
SDLC	Software Development Life Cycle
Admin	Administrator, a user role/class with system-wide management privileges
Bank Officer	A user role/class representing a bank staff member who performs customer-facing transactions (also referred to as a teller)
Customer	A user role/class representing a person who holds one or more accounts at the bank
FR	Functional Requirement
NFR	Non-Functional Requirement
CRUD	Create, Read, Update, Delete
UI	User Interface
API	Application Programming Interface
DB	Database
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
Account Type	A category of account (e.g., Savings, Current) that defines the applicable minimum balance and interest rate
Minimum Balance	The lowest balance an account of a given type is permitted to hold
Atomicity	The property that a group of operations either all take effect or none do
Audit Trail	An append-only record of every balance-changing operation and the user who performed it
INR	Indian Rupee, the single currency supported by the system

Appendix B: Analysis Models

An Entity-Relationship (ER) diagram and the corresponding relational schema for the system have been developed separately as part of the project's design artifacts and are maintained in the project's GitHub repository (design/ directory). The core entities and their relationships are summarised below for quick reference; refer to the repository artifacts for full attribute lists, keys, and cardinalities.

Entity	Related To	Nature of Relationship
Customer	Account	One-to-many: a customer may hold multiple accounts; each account belongs to exactly one customer (no joint accounts in Version 1.0).
Account Type	Account	One-to-many: each account type applies to many accounts; each account has exactly one type, which determines its minimum balance and interest rate.

Entity	Related To	Nature of Relationship
Account	Transaction	One-to-many: each account has many transaction entries; each entry belongs to exactly one account.
Transaction	Transaction (transfer pair)	One-to-one: a fund transfer is represented by two linked entries, a debit and a credit, sharing a common transfer reference.
User (Staff)	Transaction	One-to-many: each balance-changing transaction records the staff user who performed it; a staff user may perform many transactions.

Appendix C: To Be Determined List

ID	Description	Related Section
TBD-1	Final technology stack: front-end framework, the C/C++ web/API layer used to expose the back-end core, and the database product.	2.5 Design and Implementation Constraints
TBD-2	Exact set of supported account types and, for each, the minimum balance and interest rate.	2.7 Assumptions and Dependencies
TBD-3	Value of the daily withdrawal limit and whether it varies by account type.	4.4 Deposit and Withdrawal Transactions
TBD-4	Hashing algorithm and library to be used for storing passwords and PINs.	5.3 Security Requirements
TBD-5	Interest period and the basis on which interest is computed (e.g., closing balance or average balance).	4.7 Interest Computation and Account Statements
TBD-6	Hosting/deployment environment for the completed system (local, cloud, or institutional server).	2.4 Operating Environment

Appendix D: Future Enhancements

The following features were considered but are explicitly excluded from Version 1.0. They may be considered in future iterations of the system:

- Loan and EMI management
- Fixed and recurring deposit accounts
- Cheque processing and clearing
- ATM and debit card operations
- Internet banking and a native mobile application
- Interbank transfers (NEFT / RTGS / UPI)
- Multi-branch and multi-currency operation
- Joint accounts and nominee management
- Customer-initiated transfers without Bank Officer involvement
- Two-factor authentication / single sign-on

- Concurrent multi-terminal operation with record-level locking